QAs

Wednesday, July 12, 2017 9:19 AM

2071.9.
9.) What is TCP socket? Differentiate it with UDP socket. (2+3)

2072.6.
6) What are the benefits of using swing components? Explain.

2072.10.
10)What is servlet? Discuss its life cycle

2072.12.
12)What is RMI? How is it different from CORBA?

Java's Remote Method Invocation (commonly referred to as RMI) is used for client and server models. RMI is the object oriented equivalent of RPC (Remote procedure call). The Java Remote Method Invocation (RMI) system allows an object running in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. RMI provides for remote communication between programs written in the Java programming language. RMI differs from CORBA (Common Object Request Broker Architecture) in the sense that CORBA is language/machine independent whereas RMI is designed for machines running JVM, i.e. JVM dependent. The advantage of using RMI is that some operations can be executed significantly faster on the remote system than on the local client computer

DOA, dos

Remote Method invocation RMI	Common Object Request broker Arche CORBA
1) RMI interfaces are defined in java.	1) CORBA interfaces are defined in IDL.
2) RMI model represents a distributed object application.	2) CORBA is a Standard Architecture for distributed object system.
3) RMI objects are garbage collected Automatically.	3) CORBA are not garbage collected.
4) Simple to use	4) Harder to use.
5) Performance is not good, depending on application.	5) performance is good.
6) Designed for a Single language.	6) Designed for language in dependence.

Comparision of RMI & CORBA :

- RMI is suitable for small distributed applications where scalability, architecture, heterogenity & extensibility is not major concern.
- CORBA is preferred for developing full range of distributed system. eg :- Object Management Architecture.
- The RMI is inappropriate for system architecture that require container resources, load balancing or other framework functionality without custom development.
- CORBA enables interoperability between systems written in different languages & different OS and hardware plat form.

2073.9.

9. Discuss the process of sending email messages using Java

```
package com.company.bhishanwork;

import java.net.PasswordAuthentication;
import java.util.Properties;
import javax.mait.Session;
import javax.mail.PasswordAuthentication;
import javax.mail.Message;
import javax.mail.Transport;
import javax.mail.internet.InternetAddress;
import javax.mail.internet.MimeMessage;

public class EMAILJAVA {
	public static void main(String[] args) throws Exception{
		final String username = "bbhishan@mail.com" ;
		final String password = "pass" ;
		Properties props = new properties();
		props.put("mail.smtp.auth", true);
		props.put("mail.smtp.starttls.enable", true);
		props.put("mail.smtp.host", "smtp.google.com");
		props.put("mail.smtp.port", "587");

		Session session = Session.getInstance(props, new javax.mail.Authenticator{
			protected passwordAuthentication getPasswordAuthentication(){
				return new PasswordAuthentication(username, password);
			}
		});

		Message message = new Mimemessage(session);
		message.setFrom(new InternetAddress("a@a.com"));
		message.setRecipients(Message.RecipientType.To, InternetAddress.parse("b@b.com"));
		message.setSubject("Hello");
		message.setText("Hey test");
		Transport.send(message);
	}
}
```

//ALTERNATE

```
import java.util.*;
import javax.mail.*;
import javax.mail.internet.*;
import javax.activation.*;

public class SendEmail
{
 public static void main(String [] args){
     String to = "sonoojaiswal1988@gmail.com";//change accordingly
     String from = "sonoojaiswal1987@gmail.com";change accordingly
     String host = "localhost";//or IP address

    //Get the session object
     Properties properties = System.getProperties();
     properties.setProperty("mail.smtp.host", host);
     Session session = Session.getDefaultInstance(properties);

    //compose the message
     try{
        MimeMessage message = new MimeMessage(session);
        message.setFrom(new InternetAddress(from));
        message.addRecipient(Message.RecipientType.TO,new InternetAddress(to));
        message.setSubject("Ping");
        message.setText("Hello, this is example of sending email  ");

        // Send message
        Transport.send(message);
        System.out.println("message sent successfully....");

     }catch (MessagingException mex) {mex.printStackTrace();}
   }
}
```

2073.11.

11. What is InetAddress class? Discuss

InetAddress class is used to represent an IP address.
IP address can be 32 or 128 bits.

Methods:
public static InetAddress getByName(String host): returns an instance of InetAddress for the given host.
public static InetAddress getByName(byte[] addr): returns an instance of InetAddress for the given addr.
public String getHostName(): returns host name.
public String getHostName(): returns host address.
public static InetAddress getLocalhost(): returns an instance of InetAddress representing localhost.

Example:

```
import java.net.*;
public class InetAddressClass {
    public static void main(String[] args) throws Exception{
        InetAddress inetAddress = InetAddress.getByName("www.google.com");
        System.out.println("Host name = " + inetAddress.getHostName());
        System.out.println("IP address = " + inetAddress.getHostAddress());
    }
}
```

SAQs

Wednesday, July 12, 2017 9:23 AM

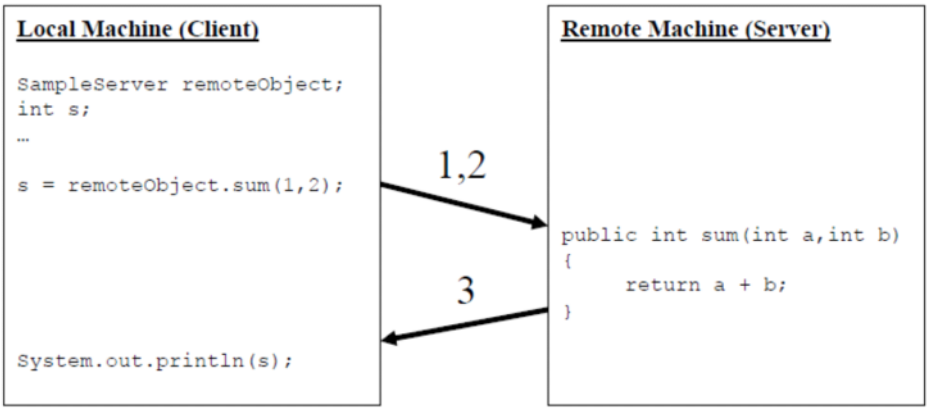
Write Short Notes on:

a. **Multithreading**

Multithreading is the ability to run multiple threads concurrently. Thus multithreaded programs can do many things at once. A thread is a separate unit of execution performing a particular task in a multithreaded program. A thread is implemented by a particular method in programming language such as Java. With multithreading, we can achieve multitasking. If we have multiple CPUs, then each thread of a program can run on different CPUs allowing parallelism. If we have only one CPU, then the threads take turns in their run and this context-switching of threads takes less time than multiprocessing systems. For example, in word processors such as MS-Word, one thread may be receiving input, another thread may be performing grammar check, and another thread may be autosaving the data, and so on. While one thread is waiting, another thread is scheduled to run in single CPU systems. Further, in multithreading, all the threads of a program share the same memory space whereas if it has been multiprocessing system, then each process would have its own memory space. Thus, multithreading is important in programming for achieving multitasking and for optimizing resource use.

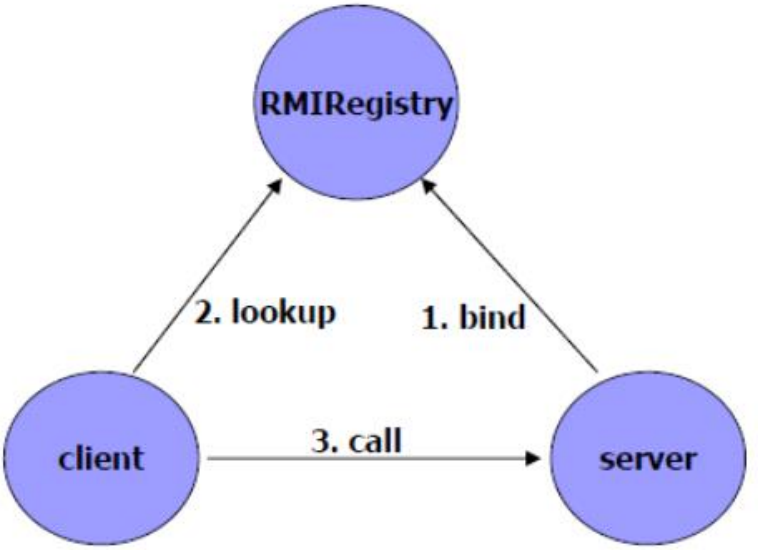
b. **RMI**

Java's Remote Method Invocation (commonly referred to as RMI) is used for client and server models. RMI is the object oriented equivalent of RPC (Remote procedure call). The Java Remote Method Invocation (RMI) system allows an object running in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. RMI provides for remote communication between programs written in the Java programming language. RMI differs from CORBA (Common Object Request Broker Architecture) in the sense that CORBA is language/machine independent whereas RMI is designed for machines running JVM, i.e. JVM dependent. The advantage of using RMI is that some operations can be executed significantly faster on the remote system than on the local client computer



c. **RMI Architecture**

To send a message to a remote server object, the client object has to find the remote object. Remote objects are listed in the RMI Registry. The Registry is a program that binds remote objects with a textual name and runs on a known port (default 1099). The RMI Registry service can be started within the server JVM, via the LocateRegistry.createRegistry() API. The client, before performing invoking a remote method, must first lookup the registry to obtain access to the remote object. After that, the client can call methods on the remote objects



A remote object is one whose instances can be used remotely. There are two parts to defining a remote class: its interface and the actual class itself. The interface definition must be public and it must extend the interface java.rmi.Remote. Further every method in the interface must declare that it throws java.rmi.RemoteException. The Remote class must implement a Remote interface and it should extend java.rmi.server.UnicastRemoteObject class. Objects of this class exist in the address space of the server and can be invoked remotely. Objects in the remote class may have methods that are not in its remote interface. However, these can only be invoked locally. The interface must be available to both client and server. The class should only be on the server.

(Code Example in 2070.3. solution)

d. Interface

Interfaces are syntactically similar to classes but they lack instance variables, and their methods are declared without any body. In practice, this means you can define interfaces which do not make assumptions about how they are implemented. Once it is defined, any number of classes can implement any number of interfaces. Using interface, you can specify what a class must do, but now how it does it.

To implement an interface, a class must create the complete set of methods defined by the interface. However, each class is free to determine the details of its own implementation. By providing the interface keyword, Java allows you to fully utilize `one interface, multiple methods` aspect of polymorphism. Interfaces are designed to support dynamic method resolution at run time. Normally, in order for a method to be called from one class to another, both classes need to be present in compile time so that Java compiler can check and ensure that the method signatures are compatible.

An interface is defined much like a class. This is the general form of an interface.

access interface *name*{

return-type method-name1(parameter-list);
return-type method-name2(parameter-list);
type final-varname1 = value;
type final-varname2 = value;
//..

return-type method-nameN(parameter-list);
type final-varnameN = value;
}

Here, **access** is either **public** or, **not used**. When no access specifier is included, then default access results, and the interface is only able to other members of the package in which it is declared. When it is declared as public, the interface can be used by any other code. **name** is the name of the instance, and can be any valid identifier.

Notice that the methods which are declared have no bodies. They end with a semicolon after the parameter list. They are, essentially, abstract methods; there can be no default implementation of any method specified within an interface. Each class that includes an interface must implement all of the methods.

Variables can be declared inside of interface declarations. They are implicitly final and static, meaning they cannot be changed by implementing class. They must also be initialized with a constant value. All methods and variables are implicitly public if the interface, itself, is declared as public.

(for Interface Implementation guidelines and example code, read Navin K Sharma Notes Unit-1 page-20)

e. Servlet

Servlets are small programs that execute on the server side of a Web connection. Just as applets dynamically extend the functionality of a Web browser, servlets dynamically extend the functionality of a Web server.
A servlet is a Java programming language class used to extend the capabilities of servers that host applications accessed via a request-response programming model. Although servlets can respond to any type of request, they are commonly used to extend the applications hosted by Web servers. For such applications, Java Servlet technology defines HTTP-specific servlet classes.The javax.servlet and javax.servlet.http packages provide interfaces and classes for writing servlets. All servlets must implement the Servlet interface, which defines life-cycle methods.

f. Lifecycle of a Servlet

Three methods are central to the life cycle of a servlet. These are init( ), service( ), and destroy( ). They are implemented by every servlet and are invoked at specific times by the server. Let us consider a typical user scenario to understand when these methods are called.

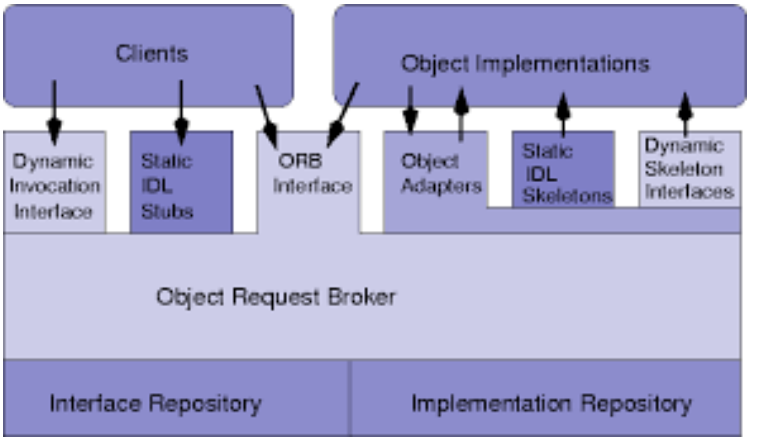
- **i.** First, when a user enters a Uniform Resource Locator (URL) to a Web browser. The browser then generates an HTTP request for this URL. This request is then sent to the appropriate server.
- **ii.** Second, this HTTP request is received by the Web server. The server maps this request to a particular servlet. The servlet is dynamically retrieved and loaded into the address space of the server.
- **iii.** Third, the server invokes the init( ) method of the servlet. This method is invoked only when the servlet is first loaded into memory. It is possible to pass initialization parameters to the servlet so it may configure itself.
- **iv.** Fourth, the server invokes the service( ) method of the servlet. This method is called to process the HTTP request. It is possible for the servlet to read data that has been provided in the HTTP request. It may also formulate an HTTP response for the client. The servlet remains in the server's address space and is available to process any other HTTP requests received from clients. The service( ) method is called for each HTTP request.
- **v.** Finally, the server may decide to unload the servlet from its memory. The server calls the destroy( ) method to relinquish any resources such as file handles that are allocated for the servlet. Important data may be saved to a persistent store. The memory allocated for the servlet and its objects can then be garbage collected.

g. **The Servlet API** *(read Navin K Sharma Notes Unit-7 page-1)*
h. **The javax.servlet Package** *(read Navin K Sharma Notes Unit-7 page-2)*
i. **The Servlet Interface** *(read Navin K Sharma Notes Unit-7 page-3)*
j. **CORBA**

The Common Object Request Broker Architecture(CORBA) is a standard defined by the Object Management Group(OMG) designed to facilitate the communication of systems that are deployed on diverse platforms. CORBA enables collaboration between systems on different operating systems, programming languages, and computing hardware. CORBA uses an object-oriented model although the systems that use CORBA do not have to be object-oriented. CORBA is an example of the distributed object paradigm.

The CORBA Interface Definition Language, or IDL, allows the development of language and location-independent interfaces to distributed objects. Using CORBA, application components can communicate with one another no matter where they are located, or who has designed them. CORBA provides the location transparency to be able to execute these applications. CORBA is often described as a "software bus" because it is a software-based communications interface through which objects are located and accessed. The illustration below identifies the primary components seen within a CORBA implementation.

k. **Architecture of CORBA**



CORBA is composed of five major components: ORB, IDL, dynamic invocation interface(DII), interface repositories (IR), and object adapters (OA)

- **ORB Interface:** Contains functionality that might be required by clients or servers.
- **Dynamic Invocation Interface (DII):** Allows client applications to use server objects without knowing the type of those objects at compile time. Used for dynamically invoking CORBA objects that were known at implementation time.
- **Dynamic Skeleton Interface (DSI):** Helps to implement generic CORBA servant.
- **Interface Repository:** Registry of fully qualified interface definitions that provide type information necessary to issue requests using the DII.
- **Dynamic Skeleton Interface:** Allows servers to be written without having skeletons, or compile-time knowledge, for the objects being implemented.
- **Object Request Broker (ORB):** Communication infrastructure for objects to communicate across the

network.

- **IDL Compiler:** Translates the IDL specifications into IDL stubs (client-side), IDL skeletons (server-side) in the desired programming language.

- **Object Adapter:** Primary means for an object implementation to access ORB services.

l. CORBA services

The CORBA standards define some standard services that may be provided to support distributed object oriented applications.There are sevral CORBA servives they are as follows:

a. **Naming Services:** Defines how CORBA objects can have friendly symbolic names
b. **Events Services:** Decouples the communication between distributed objects
c. **Object life cycle:** Defines how CORBA objects are created, removed,moved,and copied.
d. **transaction:** Coordinate atomic access to CORBA objects.
e. **concurrency:** provides a locking service for CORBA objects in order to ensure serializable access.
f. **property:** Supports the association of name-value pairs with CORBA objects
g. **security:** Mechanisms for secure channels, authorization, and auditing
h. **trader:** Supports the finding of CORBA objects based on properties describing the service offered by the object
i. **time:** Provides the current time within specified error margins.

m. Pros and Cons of CORBA

a. **PROS**
   i. Language independence
   ii. OS independence
   iii. Viable integration technology for heterogeneous legacy systems
   iv. Strong data typing

b. **CONS**
   i. Complex standard, difficult to implement for CORBA product vendors
   ii. Too complex to use compared to the features and services it provides
   iii. “Firewall-unfriendly” because CORBA(IIOP) uses arbitrary TCP ports
   iv. Lack of security

n. Features of Java

a. Simple
b. Object-Oriented
c. Platform independent
d. Secured
e. Robust
f. Architecture neutral
g. Portable
h. Dynamic
i. Interpreted
j. High Performance
k. Multithreaded
l. Distributed

Misc Topics

Wednesday, July 12, 2017 12:12 PM

Q. How do you implement MVC pattern in swing? Discuss.

Model delegate architecture. view ra controller lai combine garne

Swing API is a set of extensible GUI Components to ease the developer's life to create JAVA based Front End/GUI Applications. It is build on top of AWT API and acts as a replacement of AWT API, since it has almost every control corresponding to AWT controls. Swing component follows a Model-View-Controller architecture to fulfill the following criterias.

- A single API is to be sufficient to support multiple look and feel.
- API is to be model driven so that the highest level API is not required to have data.
- API is to use the Java Bean model so that Builder Tools and IDE can provide better services to the developers for use.

MVC Architecture

Swing API architecture follows loosely based MVC architecture in the following manner.

- Model represents component's data.
- View represents visual representation of the component's data.
- Controller takes the input from the user on the view and reflects the changes in Component's data.
- Swing component has Model as a seperate element, while the View and Controller part are clubbed in the User Interface elements. Because of

- Swing component has Model as a seperate element, while the View and Controller part are clubbed in the User Interface elements. Because of which, Swing has a pluggable look-and-feel architecture.

Swing Features

- **Light Weight** – Swing components are independent of native Operating System's API as Swing API controls are rendered mostly using pure JAVA code instead of underlying operating system calls.
- **Rich Controls** – Swing provides a rich set of advanced controls like Tree, TabbedPane, slider, colorpicker, and table controls.
- **Highly Customizable** – Swing controls can be customized in a very easy way as visual apperance is independent of internal representation.
- **Pluggable look-and-feel** – SWING based GUI Application look and feel can be changed at run-time, based on available values.

Q. Functions of CORBA Applications

12.8 The functions of CORBA Applications

In a CORBA application, the client application requests the Naming service for reference to an object. A reference to a CORBA object is returned and passed to the stub. The client invokes the method through the stub references. The stub in turn passes request to the skeleton reference obtained through the Naming service. The functioning of a CORBA application is illustrated in the figure given below:

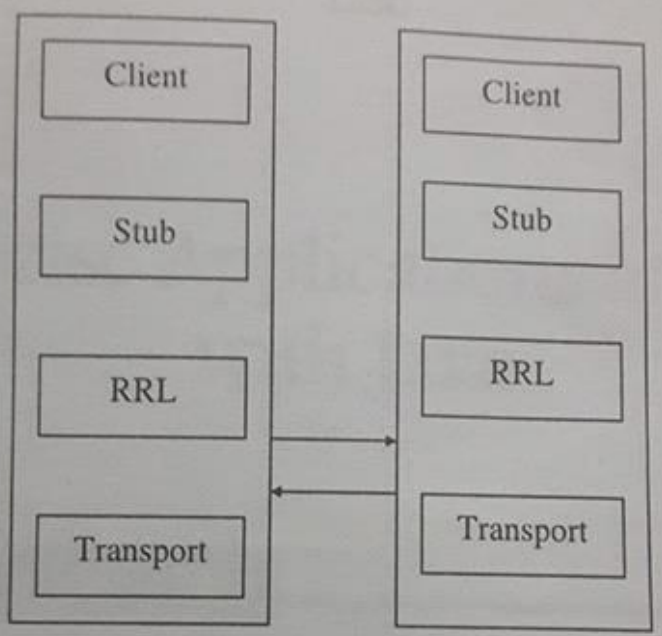


Figure 12.5: Functioning of CORBA application

The first step to design a CORBA application is to create an interface Definition language (IDL) file. We specify the functionality of CORBA application in this file. The IDl file contains the IDL interfaces, which when compiled generate the interface definition files, aserver application, The client stub, and the server skeleton. We need to save the IDl interface with the .idl extension.. The IDL interfaces are compiled using an IDl compiler.

Exercise

1. What is RMI? Discuss stub and skeleton. Explain its role in creating distributed applications.
2. Discuss RMI architecture with suitable diagram and discus each layers in the architecture briefly.
3. Explain the steps involved in creating RMI applications with suitable java code.

Q. Difference between TCP and UDP

TCP	UDP
1) Transfer Control Protocol	1) User data gram protocol
2) is Connection-oriented protocol.	2) Connection less protocol
3) Data is read as "stream" with nothing distinguishing where one packets ends and other begins. There is guarentee of transfering data in order.	3) Data is sent independently in the form of packets called datagrams. There is no guarentee of transfering data in order.
4) Used by other protocols such as HTTP, HTTPS, FTP, SMTP, Telnet.	4) by DNS, DHCP, RIP.
5) Speed of transfer is slow.	5) fast.
6) Guarentee of transfer of data.	6) Not guarentee
7) Header size is 20 bytes.	7) Header size is 8 bytes.
8) Heavy weight.	8) Light weight.
9) consist Acknowledgement segment	9) No acknowledgement
10) Handshake SYN, SYN-ACK, ACK	10) No Handshake.

Programs:

Wednesday, July 12, 2017 9:41 AM

2069.13. 2070.13. 2071.13.
13. Write a simple JSP program to display "Lalitpur, Nepal" 10 times. (5)

The JSP program code is given below. It utilizes scriptlets to perform the task.

```
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>A simple JSP program</title>
    </head>
    <body>
        <h1>Displaying "Lalitpur, Nepal" 10 times!</h1>
        <table>
            <%
            for(int i=1; i<=10; i++) {
            %>
            <tr><td>Lalitpur, Nepal</td></tr>
            <% } %>
        </table>
    </body>
</html>
```

2072.4. 2073.4.
4. Write an object oriented program to find the area and perimeter of rectangle

2072.5. 2073.5.
5. Write the simple java program that reads data from one file and writes data to another file.

Model.
Send Emails using Java

```
package com.company.bhishanwork;

import java.net.PasswordAuthentication;
import java.util.Properties;
import javax.mait.Session;
import javax.mail.PasswordAuthentication;
import javax.mail.Message;
import javax.mail.Transport;
import javax.mail.internet.InternetAddress;
import javax.mail.internet.MimeMessage;
```

```
public class EMAILJAVA {
	public static void main(String[] args) throws Exception{
		final String username = "bbhishan@mail.com" ;
		final String password = "pass" ;
		Properties props = new properties();
		props.put("mail.smtp.auth", true);
		props.put("mail.smtp.starttls.enable", true);
		props.put("mail.smtp.host", "smtp.google.com");
		props.put("mail.smtp.port", "587");

		Session session = Session.getInstance(props, new javax.mail.Authenticator{
			protected passwordAuthentication getPasswordAuthentication(){
				return new PasswordAuthentication(username, password);
			}
		});

		Message message = new Mimemessage(session);
		message.setFrom(new InternetAddress("a@a.com"));
		message.setRecipients(Message.RecipientType.To, InternetAddress.parse("b@b.com"));
		message.setSubject("Hello");
		message.setText("Hey test");
		Transport.send(message);
	}
}

//ALTERNATE

	import java.util.*;
	import javax.mail.*;
	import javax.mail.internet.*;
	import javax.activation.*;

	public class SendEmail
	{
	 public static void main(String [] args){
		String to = "sonoojaiswal1988@gmail.com";//change accordingly
		String from = "sonoojaiswal1987@gmail.com";change accordingly
		String host = "localhost";//or IP address

		//Get the session object
		Properties properties = System.getProperties();
		properties.setProperty("mail.smtp.host", host);
		Session session = Session.getDefaultInstance(properties);

		//compose the message
		 try{
		   MimeMessage message = new MimeMessage(session);
		   message.setFrom(new InternetAddress(from));
		   message.addRecipient(Message.RecipientType.TO,new InternetAddress(to));
		   message.setSubject("Ping");
		   message.setText("Hello, this is example of sending email  ");

		   // Send message
		   Transport.send(message);
		   System.out.println("message sent successfully....");
```

```
        }catch (MessagingException mex) {mex.printStackTrace();}
    }
}
```

From <https://www.javatpoint.com/example-of-sending-email-using-java-mail-api>

Q. RMI to develop program that runs in different machine

```
/* RMI to develop program that runs in different machine
 *
 * STEPS to run this program
 * 1. Open terminal and move to directory containing these files
 * 2. Compile all files
 *                    javac *.java
 * 3. Run the following commands
 *                    rmic AdderRemote
 *                    rmiregistry 1234
 * 4. Open second terminal and start server
 *                    java Server
 * 5. Open third terminal and start client
 *                    java Client
 */

//Adder Interface : AdderInterface.java
import java.rmi.*;

public interface AdderInterface extends Remote {
   public int add(int x, int y)throws RemoteException;
}

//Adder Remote : AdderRemote.java
import java.rmi.*;
import java.rmi.server.*;

public class AdderRemote extends UnicastRemoteObject implements AdderInterface {
   AdderRemote()throws RemoteException {
      super();
   }

   public int add(int x, int y) {
      return x+y;
   }
}

//Server : Server.java
import java.rmi.*;

public class Server{
   public static void main(String args[]) {
      try {
         AdderInterface stub = new AdderRemote();
         Naming.rebind("rmi://localhost:1234/sagunsh", stub);
      } catch(Exception e) {
         System.out.println(e);
```

```
        }
    }
}

//Client : Client.java
import java.rmi.*;

public class Client{
    public static void main(String args[]) {
        try {
            AdderInterface stub = (AdderInterface) Naming.lookup("rmi://localhost:1234/sagunsh");
            System.out.println(stub.add(103,51));
        } catch(Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Bean Writing Process

Wednesday, July 12, 2017 2:41 PM

Property sheet.

3. Edit the label field in the button's Property sheet so that the button's label reads "start."
4. Use the BeanBox edit menu to select an action event to be fired by the start button. Before choosing the event action, be sure that you have selected the start button.

Next mouse press should be over the target Bean which defines appropriate event-handler methods, in this case the Juggler Bean. For this, we need to follow following steps:

1. Drag the line from the start button and release it over the Juggler Bean. A dialog appears listing applicable event handlers defined by the Juggler Bean.
2. Select the start Juggling method as the target for the event, and then press OK.

Now, when we press the start button Duke should start tossing beans around in a circle over his head like a professional juggler.

Completing the Program

To complete the example program, add a stop button. Repeat the steps that are performed for adding the start button and connect it to the appropriate Juggler action. Now, we should be able to start and stop the juggler by pressing the appropriate button.

9.4 Bean Writing Process

Writing a bean is not technically difficult—there are only a few new classes and interfaces for us. In particular, the simplest kind of a bean is nothing more than a Java class that follows some fairly strict naming conventions for its methods. Steps for writing a bean are:

1. create a directory for the new Bean
2. create the java source file(s)
3. compile the source file(s)
4. Create manifest file.
5. Generate a JAR file.
6. Start the BDK.
7. Test the bean

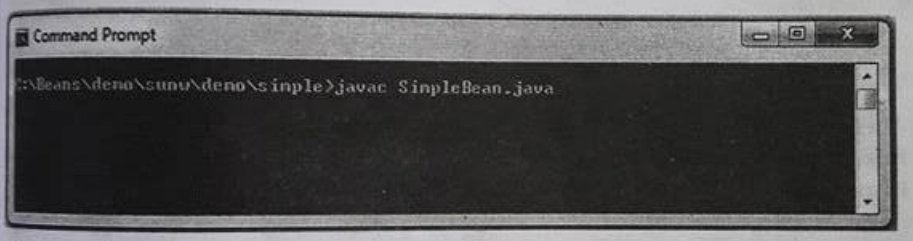
Create a directory for the new Bean

In our example C:\Beans\demo\sunw\demo\simple

Create the java source file(s)

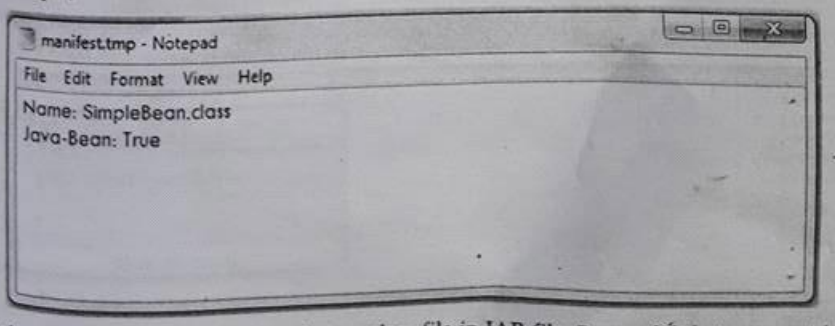
```
import java.awt.*;
import java.io.Serializable;
public class SimpleBean extends Canvas implements Serializable
{
    public SimpleBean()
    {
        setSize(60,40);
        setBackground(Color.red);
    }
}
```

Compile the source file(s)



Create manifest file

We must create the manifest file that provides information about which components in the JAR file are java beans. The manifest file must have name *manifest.tmp* or *menifest.mft*. We can put any number of files in the manifest file entry but if we have class files acting as java beans then each entry of the file must be immediately followed by the line **Java-Bean: True.** For example,



The above file indicates there is only one class file in JAR file. Be careful that we must keep new line character (i.e. press enter key) after second entry.

Generate a JAR file

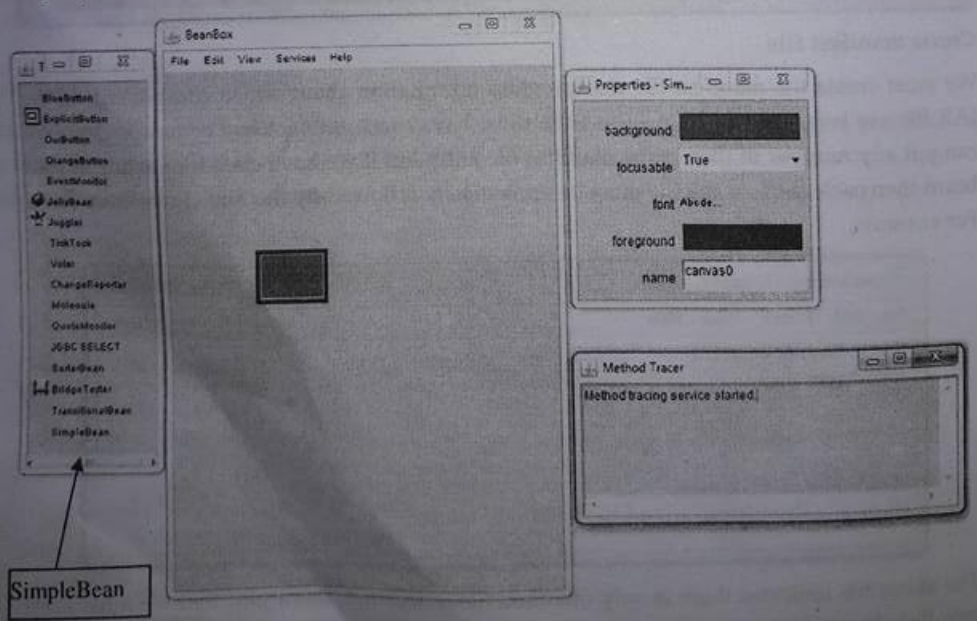
Use Jar tool provided with JDK to create JAR file. We must include all classes and manifest file in the JAR file as below:

Start the BDK and Load the Beans JAR File

Start the BDK and click on *File* menu of BeanBox, then click on *Load JAR....* Menu item and select the JAR file that is created in previous step. If we want that the bean created recently should be loaded automatically when we start BDK, we must put the JAR file in the folder *C:\Bean\Jars*

Test the Bean

After the bean is started, click the SimpleBean that appears in Toolbox and drop it into BeanBox. Now we can customize this bean by changing different properties appeared in Property box.



Java Beans vs Class

Wednesday, July 12, 2017 2:43 PM

Following features of bean separate from class
-default , no argument constructor
-setter and getter methods
-serializable interface

Beans ma
properties haru private huncha
setter ra getter huna parcha
implements java.io.Serializable
Default constructor (no arguments)

Normal java class ma default more than one constructor banaucham ni tannai parameter halera.. Beans MA chai parameterized constructor paidaina.. Yesko matlab pachi object banaisakepachi setter bata set garincha teskaran setter getter compulsory vako ho

From <https://www.facebook.com/groups/csitbatch2070/?multi_permalinks=1508362072532744&notif_t=group_activity&notif_id=1499849317661365>

Layout Management

Wednesday, July 12, 2017 3:08 PM

Border Layout -

A border layout lays out a container arranging and resizing its components to fit in five regions: North, South, East, west and center. Each region contains no more than one component, and is identified by corresponding constant: NORTH, SOUTH, EAST, WEST and CENTER. When adding a components to a container with this layout, we have to use one of these five constants. Example.

```
import java.awt.BorderLayout;
import javax.swing.*;
public class BorderTest extends JFrame{
JButton north, south, east, west, center;
 public BorderTest()
 { setLayout(new BorderLayout());
 north = new JButton("NORTH");
 south = new JButton("South");
 center = new JButton("Center");
add(north, BorderLayout.NORTH);
add(center, BorderLayout.CENTER);
setSize(300, 300);
 setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
public static void main(String[] args){
    new BorderTest();
}
```

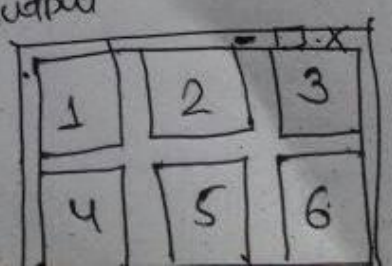
GridLayout is a layout manager that lays out a container's components in a rectangular grid. The container is divided into equal sized rectangles, and one component is placed in each rectangle. Thus we can say that the components will be arranged in the form of rows and columns like a table with equal sized cells. The constructors for GridLayout are:-

a) GridLayout()
b) GridLayout (int rows, int cols)
c) GridLayout (int rows, int cols, int hgap, int vgap)

Example:-

```
import java.awt.GridLayout;
import javax.swing.*;
public class GridTest extends JFrame{
    public GridTest(){
      setLayout(new GridLayout(2,3,10,20));
      JButton one = new JButton("1");
      JButton two = new JButton("2");
      :
      JButton six = new JButton("6");
    add(one); add(Two); ... add(six);
    setSize (300, 300);
    setVisible (true);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }
  public static void main (String[] args){
      new GridTest();
  }
}
```

output



JSP vs Servlet

Wednesday, July 12, 2017 4:05 PM

JSP	Servlets
JSP is a webpage scripting language that can generate dynamic content.	Servlets are Java programs that are already compiled which also creates dynamic web content.
JSP run slower compared to Servlet as it takes compilation time to convert into Java Servlets.	Servlets run faster compared to JSP.
It's easier to code in JSP than in Java Servlets.	Its little much code to write here.
In MVC, jsp act as a view.	In MVC, servlet act as a controller.
JSP are generally preferred when there is not much processing of data required.	servlets are best for use when there is more processing and manipulation involved.
The advantage of JSP programming over servlets is that we can build custom tags which can directly call Java beans.	There is no such custom tag facility in servlets.
We can achieve functionality of JSP at client side by running JavaScript at client side.	There are no such methods for servlets.